GUÍA GENERAL Y ENUNCIADO OFICIAL

Práctica: Calculadora en JavaScript

✍️ Por Ramón Abramo • Profesor de Desarrollo Web

¡Buenas a todos! Soy Ramón Abramo. Bienvenidos a la guía general de la práctica de la calculadora. He diseñado este material para estructurar el aprendizaje de mis alumnos de forma clara, amena y rigurosa. En este documento tenéis el **enunciado oficial**, los **7 hitos de desarrollo** y el análisis de las **dos soluciones técnicas** que he preparado para vosotros.

🎯 1. ¿Por qué una calculadora en JavaScript?

Siempre que iniciamos el bloque de interactividad en el aula, les digo a mis alumnos: *"Si sois capaces de programar una calculadora completa sin bugs, dominái el 90% de la lógica que os pedirán en una empresa para manipular el DOM"*.

🖥️ El DOM
Capturar nodos, modificar `textContent` y reaccionar a pulsaciones en tiempo real sin recargar la página.

🧠 El Estado
Aprender a guardar datos en la memoria volátil del programa: operando actual, previo, operador y banderas de control.

⚡ Dos Soluciones
Compararemos la solución básica (16 escuchadores individuales) frente a la profesional (delegación con Event Bubbling).

📂 2. Organización del Repositorio de Prácticas

Para que no haya confusiones en el aula, he organizado el proyecto en dos carpetas totalmente independientes y autónomas:

- `Guia_General_Calculadora_JS_Ramon_Abramo.pdf` : Este documento con la visión global.
- `README.md` : El índice markdown con el enunciado y cuadro comparativo.
- `version-1-escuchadores-individuales/` : Solución inicial con un listener por botón. Incluye su propio PDF detallado.
- `version-2-delegacion-eventos/` : Solución avanzada con 1 solo listener en el contenedor padre. Incluye su propio PDF detallado.

📋 Enunciado del Ejercicio para el Alumnado

> **Objetivo:** Implementar en JavaScript toda la lógica interactiva de una calculadora de cuatro operaciones básicas (+, -, *, /), memoria de operandos, encadenamiento de cálculos y borrado total.

1. Anatomía del HTML Provisto (`index.html`)

El diseño visual ya está terminado con estética retro y display LCD. Fijaos en la semántica de las etiquetas y atributos de datos:

```
<!-- Pantalla LCD donde pintaremos números y resultados -->
<output id="display" class="display">0</output>

<!-- Contenedor padre de todos los botones -->
<div class="teclado">
  <button class="tecla tecla--borrar" data-accion="borrar">C</button>
  <button class="tecla tecla--operador" data-operador="/">/</button>
  <button class="tecla" data-numero="7">7</button>
  <button class="tecla" data-numero="8">8</button>
  <button class="tecla" data-numero="9">9</button>
  <button class="tecla tecla--operador" data-operador="*">*</button>
  ...
  <button class="tecla tecla--igual" data-accion="igual">=</button>
</div>
```

Requisitos Obligatorios del Sistema

Entrada de Datos

- Los dígitos tecleados sustituyen al `0` inicial (evitando `"07"`).
- Concatenación correcta al teclear múltiples cifras (ej: `"1"` + `"2"` = `"12"`).
- Tras presionar una operación, el siguiente dígito limpia la pantalla.

Operaciones Matemáticas

- Las cuatro operaciones básicas: Suma, Resta, Multiplicación y División.
- Botón igual (`=`) calcula y muestra el resultado.
- Operaciones encadenadas sin pulsar `=` (ej: `5 + 3 + 2 =` `10`).

⚠️ Control de Excepciones y Robustez

1. **División por cero:** Si el usuario divide cualquier cifra entre 0, la calculadora no debe mostrar `Infinity`, sino el mensaje `Error`.
2. **Redondeo de coma flotante:** Es obligatorio corregir las imprecisiones de JavaScript (como `0.1 + 0.2 =` `0.30000000000000004`).
3. **Tecla 'C':** Debe devolver toda la calculadora a su estado de fábrica en cualquier momento.

🚀 Desarrollo Incremental: Los 7 Hitos

> Para que los alumnos no se sientan perdidos, el ejercicio se resuelve de forma **secuencial por hitos**. Cada hito añade una funcionalidad visible y comprobable en el navegador.

Hito	Objetivo Funcional	Lógica Interna	🧪 Prueba en Clase
HITO 1	**Escribir números**	Controlar click en 0-9. Si hay '0' se sustituye; si no, se concatena texto.	Pulsar `1, 2, 3` ➔ Muestra `123`.
HITO 2	**La Suma (+) y Memoria**	Guardar `operandoActual` en `operandoAnterior`, memorizar '+' y activar `reiniciarDisplay = true`.	Escribir `15 + 4` ➔ Se ve `4`.
HITO 3	**Resolver con (=)**	Convertir cadenas con `parseFloat()`, realizar la suma y mostrar el total.	Pulsar `7 + 8 =` ➔ Muestra `15`.
HITO 4	**Resta, Multiplicación, División**	Extender el cálculo con un `switch (operacion)` para -, * y /.	`9 - 4 = 5` `6 * 3 = 18`
HITO 5	**Operaciones encadenadas**	Si ya había operación previa, calcular automáticamente al pulsar el siguiente operador.	`5 + 3 +` ➔ La pantalla ya muestra `8`.
HITO 6	**Botón de Borrado (C)**	Función `borrar()` que resetea todas las variables a `'0'`, `null` y `false`.	Escribir `999` y pulsar `C` ➔ `0`.
HITO 7	**Control de errores**	División por cero da `Error` y redondeo preventivo con `Math.round`.	`9 / 0 =` ➔ `Error`.

🧠 Las 4 Variables de Estado en Memoria

Ambas soluciones comparten exactamente la misma memoria interna:

```
let operandoActual = '0';      // Lo que se ve en el display LCD
let operandoAnterior = null;   // Número previo guardado antes del operador
let operacion = null;          // Símbolo '+', '-', '*' o '/'
let reiniciarDisplay = false;  // Bandera: ¿debe el próximo número limpiar la pantalla?
```

⚖️ Dos Enfoques para Gestionar los Eventos

Para resolver este ejercicio he creado dos carpetas independientes. Cada una implementa los mismos 7 hitos pero con una arquitectura de eventos distinta:

📁 Solución 1: Escuchadores Individuales

- Seleccionamos cada grupo de botones con `querySelectorAll`.
- Usamos bucles `forEach` para asignar un `addEventListener` a cada tecla.
- Se crean **16 escuchadores en memoria**.
- **Didáctica:** Ideal para la primera semana de clase; relación 1 a 1 transparente.

📁 Solución 2: Delegación de Eventos

- Registramos un **único escuchador** en el padre `.teclado`.
- Aprovechamos el borboteo de eventos (`Event Bubbling`).
- Identificamos el botón con `e.target.closest('button')` y `dataset`.
- **Didáctica:** Nivel profesional; mínimo consumo de memoria y máxima escalabilidad.

Cuadro Comparativo Exhaustivo

Criterio Técnico	Solución 1: Individuales	Solución 2: Delegación
Carpeta de trabajo	`version-1-escuchadores-individuales/`	`version-2-delegacion-eventos/`
Escuchadores en memoria	16 escuchadores activos	**1 solo escuchador activo**
Mecanismo DOM	Vínculo directo elemento por elemento	**Event Bubbling** en el contenedor padre
Lectura de datos	Directa en cada callback	Vía `closest('button')` y `dataset`
Nuevos botones dinámicos	Requieren reconectar listeners a mano	Funcionan solos sin tocar código JS
Dificultad inicial	Baja (muy intuitiva para noveles)	Media (requiere entender el objeto evento)

💡 La Gran Lección de Software

Fijaos en que la matemática (las funciones `agregarNumero`, `seleccionarOperador`, `calcular` y `borrar`) es **exactamente idéntica** en ambas soluciones. Solo cambia la forma en que escuchamos las órdenes del usuario. Esto es la base del código desacoplado.

👩‍🏫 Cómo Impartir esta Práctica en el Aula

Basándome en mis años de docencia, os recomiendo planificar esta práctica en **dos fases pedagógicas bien diferenciadas**:

🗓️ Estrategia Didáctica en Dos Tiempos

• **Semana 1 (Asentar Bases):** Los alumnos trabajan con la **Solución 1**. Conectan botones con bucles `forEach`, comprenden las variables de estado y resuelven la lógica aritmética. El objetivo aquí es que la calculadora funcione sin bloquearse con conceptos abstractos de propagación.

• **Semana 2 (Refactorización Profesional):** Abrimos la **Solución 2**. Les planteo el reto: *"¿Cómo haríais si la calculadora tuviera 60 botones?"* Les explico el borboteo de eventos y vemos cómo con un solo listener en `.teclado` el código pasa de 16 listeners a 1. El salto cualitativo en su madurez como programadores es espectacular.

📊 Rúbrica de Evaluación para el Alumnado

Criterio	Aspectos a Evaluar	Peso
Lógica Aritmética	Cálculo exacto de +, -, *, /, encadenamiento de operaciones y redondeo flotante.	**35%**
Control del DOM	Gestión de pantalla limpia tras operador, eliminación del 0 inicial y tecla C.	**25%**
Eventos y Robustez	Correcta asignación de eventos, control de división entre 0 (muestra "Error").	**25%**
Calidad de Código	Nombres descriptivos, código modular, uso de `const` y `let` estricto.	**15%**

🚀 Materiales Adicionales Listos para Usar

En cada carpeta tenéis: **1.** `presentacion.html` (diapositivas interactivas en pantalla completa con la tecla `F`).
2. `app.alumnos.js` (plantilla con bloques `TODO` vacíos para entregar a los alumnos).
3. `app.solucion.js` (código canónico para proyectar o corregir).
4. Un PDF detallado para cada una de las soluciones.